

Driving an OSSM from video playback

How to connect BX Viewer to an OSSM through OSSM Sauce, so the machine follows the same `.bx` path the waveform is drawn from, in time with the video.

CONTENTS

- | | | | |
|---|----------------|----|--------------------------|
| 1 | What this does | 7 | Settings reference |
| 2 | What you need | 8 | Troubleshooting |
| 3 | One-time setup | 9 | Testing without hardware |
| 4 | Every session | 10 | How it works |
| 5 | Playing | 11 | Known limits |
| 6 | Tuning | | |

1 · What this does

A `.bx` file already describes where the stroke should be at every moment of a video — that is what the waveform under the player draws. Device output takes that same path and sends it to your machine as it plays, so the OSSM follows what is on screen.

It runs entirely in the browser page. There is no extra server to start and no change to your files. Playback stays in charge: pause and the machine stops, seek and it picks up wherever you landed, switch browser tabs and it halts.

THE SHORT VERSION

Tick one box in the viewer's settings. Put OSSM Sauce in **MCP** bridge mode and press Enable. Press play.

2 · What you need

An OSSM	Powered on, running OSSM Sauce firmware, and joined to your network.
OSSM Sauce	The desktop app. It is the bridge — the browser cannot reach the machine without it. Version 1.5.1 or newer, which is where the MCP bridge mode appears.
A video with a path	Any title in your library that has a <code>.bx</code> file. No path, nothing to send.
Intiface Central	<i>Not needed.</i> Only if you are driving something other than an OSSM — see §3.

WHY OSSM SAUCE IS NOT OPTIONAL

The OSSM's ESP32 is a WebSocket *client*: it dials out to OSSM Sauce, which is the server. Nothing can connect *to* the machine, so every route to it — ours included — ends up going through the app.

3 · One-time setup

In the viewer

- 1 Open **Settings** → **Device output (OSSM)**.
- 2 Tick **Enable device output**. This is the master switch, and it is off by default so that opening a page can never start hardware on its own.
- 3 Leave **Connection** on **OSSM, raw (MCP bridge)** and the address on `http://127.0.0.1:8081`. This is the default.
- 4 Tick **Connect automatically** if you want it to connect whenever a player page opens, rather than pressing Connect each time.
- 5 Save.

In OSSM Sauce

- 1 Open **Bridge Settings**, set **Bridge Mode** to **MCP**, and press **Enable**.

That is the whole setup. The bridge mode is remembered between launches; only **Enable** needs pressing each time you open the app.

WHY THIS ROUTE AND NOT THE OTHER TWO

The **XToys** and **Buttplug.io** bridges take a *description* of each move and decide for themselves what the machine does with it. Both clamp anything shorter than **Minimum Duration** — 500 ms out of the box, which stretches a dense path until the machine falls hopelessly behind — and both re-ease every stroke with **Stroke Smoothing**, which OSSM Sauce resets to **Sine** on every launch because it never saves the setting.

The MCP bridge does neither. It passes the bytes we send straight to the machine, so the viewer builds the firmware's own move command and chooses the motion directly. Nothing to set by hand, and nothing to remember to re-set tomorrow.

The other two options remain in the Connection dropdown. Pick **Intiface Central** if you are driving something other than an OSSM, or already run Intiface for other reasons; it reaches an OSSM only by way of OSSM Sauce registering itself with Intiface as a TCode device. Pick **XToys** only if the MCP mode is unavailable in your build. Both then need **Minimum Duration** at 50 ms and **Stroke Smoothing** on **Linear** — the latter every launch.

4 · Every session

- 1 **Power the OSSM on and let it connect to OSSM Sauce** before you start playback.
- 2 **Press Enable** on the bridge screen.

That is it — there is no setting to re-apply. The viewer asks OSSM Sauce every couple of seconds whether the machine is actually attached, so if you forget step 1 the Device panel will tell you rather than leaving you to wonder why nothing is moving.

ON SAFETY LIMITS

Sending the machine's own command directly does not bypass anything that protects you. Depth is expressed as a fraction of the range you configured in OSSM Sauce, and the firmware maps it into that range and clamps it; speed and acceleration limits are enforced on the machine as always. It is the same command the bridges send, with the timing and easing fields left as we set them.

5 · Playing

Open a video and press play. That is the whole operation — the machine tracks the path from there.

The **Device** panel lives at the top of the **BounceX Data** tab in the player sidebar, and the viewer opens that tab for you when device output is enabled. Connect from there if you did not turn on automatic connection. The status dot is the thing to watch:

Grey	Not connected.
Amber	Connecting, or connected but not yet armed.
Solid teal	Connected and driving.

Playback controls behave the way you would expect. Pausing stops the machine where it is. Seeking or scrubbing re-anchors it to the depth the path calls for at the new position — a jump of more than a quarter-second is treated as a seek rather than as drift. Switching to another browser tab halts it, and so does closing the page.

6 · Tuning

Three settings in **Settings** → **Device output** are worth reaching for once it is running. The first two take effect on the very next command, with no reload.

Stroke range

Limits how much of the machine's travel the path uses, as a percentage. Lower it to keep the whole path within a shallower window without changing the file.

Sync offset

Shifts the device in time against the video, in milliseconds. Positive delays the machine, negative makes it lead. Use it for mechanical lag: if the machine feels consistently a beat behind everything on screen, a negative offset of 50–150 ms usually settles it. Only use this for a *constant* error — an error that grows over the course of a video is the Minimum Duration clamp, not this.

Minimum move

The shortest command the viewer will issue; anything closer together than this gets merged. Raise it if the machine stutters or cannot keep up on dense paths — a coarser path it can actually follow beats a detailed one it cannot. This is the one tuning knob that costs something to change, because it replans the track.

7 · Settings reference

Setting	What it does
Enable device output	Master switch. Off by default; nothing connects or moves while it is off.
Connect automatically	Connects as soon as a player page opens, instead of waiting for the Connect button.
Connection	<i>OSSM</i> , <i>raw</i> is the default and goes through the app's MCP bridge untouched. <i>XToys</i> and <i>Intiface Central</i> are the older routes; the latter also drives non-OSSM positional toys.
Server address	<code>http://127.0.0.1:8081</code> for MCP, <code>ws://127.0.0.1:8080</code> for XToys, <code>ws://127.0.0.1:12345</code> for Intiface. Change only if you moved the port.
Invert direction	Swaps the ends of the stroke.
Stroke range	Percentage of available travel the path is mapped into.
Sync offset	Milliseconds of lead or lag against the video.
Minimum move	Shortest command issued; commands closer than this are merged.

And on the OSSM Sauce side

Setting	Set it to
Bridge Mode	<code>MCP</code> — then press Enable. Nothing else on this list applies to that mode.
Only if you are using the XToys or Buttplug.io bridge instead	
Speed Overrides → Minimum Duration	<code>50 ms</code> . Default 500 ms is far too slow.
Stroke Smoothing	<code>Linear</code> . Every launch — it is not saved.
Get stroke duration from command	Ticked (XToys only). Otherwise our timing is discarded.
Max Message Frequency	Ignore it. With stroke duration taken from the command it is unused; it is a duration fallback, not a rate limit, and it never drops anything.

8 · Troubleshooting

Symptom	Cause and fix
---------	---------------

Panel cannot reach the server	OSSM Sauce is not running, the bridge is not Enabled , or it is in the wrong mode. Check the bridge is on MCP and the port matches the address in Settings.
Connected, but the panel lists no device	The bridge is up and the OSSM is not attached to OSSM Sauce. Power the machine on and let the app pick it up; the panel notices within a couple of seconds.
Connects, but the machine never moves	Same cause, on the older routes — but there the bridge discards every command while the machine is absent and tells nobody, so the viewer looks perfectly healthy. Confirm the app shows the OSSM connected. (The MCP route reports this properly, which is one reason it is the default.)
Starts in sync, drifts later and later behind	On XToys or Buttplug.io, Minimum Duration is still 500 ms; set it to 50. It cannot be the cause on the MCP route, which has no such clamp.
Motion pulses, or stops and restarts constantly	On XToys or Buttplug.io, Stroke Smoothing is on Sine ; set it to Linear , and remember it resets every launch. The MCP route sets this per command and is immune.
Every stroke feels the same speed; the shape is gone	XToys only: Get stroke duration from command is unticked, so all timing is being replaced by a fixed value.
Consistently a beat behind, but not drifting	Mechanical lag. Dial in a negative Sync offset .
Stutters or falls behind on busy passages	Raise Minimum move so fewer, longer commands are sent.
Stroke feels too deep or inverted	Stroke range and Invert direction .
Panel stays grey on a video	That title has no .bx path, or device output is switched off in Settings.
Chrome asks about local network access	Expected on Chrome 147 and later — the page is opening a connection to your own machine. Allow it.
Nothing works in Safari over https	Safari blocks loopback WebSockets from a secure page outright. Use Chrome or Firefox, or serve the viewer over http://localhost , where the restriction does not apply.
Vibration-only toy shows up on Intiface but never runs	Expected. Only positional devices are driven; there is no stroke-to-vibration fallback.

9 · Testing without hardware

The repository ships a fake server that pretends to be Intiface with one linear device attached, and prints every move it receives as a position bar. It is useful for confirming the viewer end works before you involve the machine.

```
bun run sim                # listens on ws://127.0.0.1:12345
bun run sim -- --port 47000 # somewhere else
bun run sim -- --gap 50     # emulate a device with a 50 ms timing gap
```

It speaks Buttplug, so switch **Connection** to **Intiface Central** and `ws://127.0.0.1:12345` while you use it, then switch back. Stop Intiface Central first if it is running — they cannot share the port.

10 · How it works

A pair of `.bx` markers describes a *curve*: go from depth A to depth B over so many frames, following a particular easing. Essentially no device understands that. What they all accept is "be at this position in this many milliseconds, travelling straight there."

So the viewer converts the path in three steps. It reads the markers into timed curve segments, which loses nothing. It then flattens those into linear moves, subdividing only the curves that measurably deviate from a straight line and merging anything closer together than the device can accept. Finally a scheduler issues each move at the right moment against video time, handling seeks, pauses, and the occasional late frame — when a command goes out slightly late, its duration is shortened by exactly that amount so the machine still arrives on schedule.

Two consequences are worth knowing. **The easing you see is the easing the machine gets**, because the curve is flattened here using the viewer's own easing code rather than being handed to the firmware to interpret. That also sidesteps a real incompatibility: OSSM firmware numbers its transition types differently from the editor, so five of the eight values mean a different curve on each side. We never send one. And **the command rate stays low** — across a typical library, plans run three to eight commands a second, so there is nothing here that a bridge or a motor should struggle with.

On the MCP route the last step is literal: the viewer builds the ten-byte `SMOOTH_MOVE` frame the firmware expects — command byte, duration, depth on a 0–10,000 scale, transition, easing — hex-encodes it, and posts it. OSSM Sauce broadcasts those bytes to the machine without inspecting them. The transition byte is always *linear*, which is the correct choice precisely because the curve has already been flattened: each command is a straight segment, and the machine drawing it straight is what reproduces the original curve.

11 · Known limits

- **One device at a time** — the first one that reports positional support.
- **Positional devices only**. A vibration-only toy will connect and be listed, but nothing drives it.

- **The XToys bridge never reports back.** On that route the viewer can tell that OSSM Sauce is running, but not whether the OSSM is. The MCP and Buttplug routes both report device presence.
- **The MCP route is OSSM-only.** It speaks the firmware's command set directly, so unlike the Buttplug route it is not a general transport for other hardware.
- **Playback speed changes are not tracked.** The player never changes playback rate, and the timing would not follow it if it did.
- **Buttplug message spec v3.** Intiface 3.1 runs a v4 server but translates for older clients, and OSSM Sauce's own bridge connects as v3 — which makes v3 the verified-compatible choice rather than the outdated one.